memo, useMemo, useCallback e React Compiler

React Avançado · Sessão 4 · resumo e perguntas para rever

O problema de base

Em cada render, **o componente corre outra vez, de cima a baixo**. Tudo o que lá está é **criado de novo**: objetos, arrays, funções e JSX.

E objetos e funções novos são **diferentes** dos anteriores, mesmo com o mesmo conteúdo:

```
{ from: 'Lisbon' } === { from: 'Lisbon' }   // false — dois objetos, dois sítios da memória
(() => {}) === (() => {})                    // false
'Lisbon' === 'Lisbon'                        // true  — os primitivos comparam-se pelo valor
```

`memo`, `useMemo` e `useCallback` servem todos para dizer ao React: **"isto é o mesmo de antes"**.

`memo` — num componente

Antes de renderizar o componente, **compara as props novas com as anteriores**. Todas iguais → **salta o render**.

```
function FlightCardBase({ flight, selected, onToggle }: FlightCardProps) {
  // …
}
export const FlightCard = memo(FlightCardBase);
```

Pensem nele como **um porteiro à entrada do componente**: "as props são as mesmas de há pouco? Então não entras."

Atenção: compara **por referência**. Uma função ou um objeto novo em cada render do pai → o porteiro deixa sempre passar.

`useCallback` — numa função

Devolve **a mesma função** entre renders, até uma dependência mudar.

```
const updatebasket = useCallback((id: number, selected: boolean) => {
  setBasket(b => ({ ...b, [id]: selected }));
}, []);   // dependências
```

Serve para **passar a um filho com `memo`**. Sozinho, não evita render nenhum.

`useMemo` — num valor

Guarda **o resultado de um cálculo** e só o refaz quando uma dependência muda.

```
const sorted = useMemo(() => flights.toSorted(byPrice), [flights]);
//                     ──────── 1.º argumento ────────  ── 2.º ──
//                     "como calcular"                   "quando recalcular"
```

Serve para: um **cálculo pesado**, ou um **objeto/array estável** para um filho com `memo`.

	Recebe	Guarda	Devolve
`useMemo(fn, deps)`	uma função	o **resultado** de a chamar	um **valor**
`useCallback(fn, deps)`	uma função	a **própria função**	uma **função**

Como se relacionam

```
        o pai renderiza
              │
   useCallback / useMemo  →  props com a MESMA referência
              │
            memo          →  compara as props: iguais? salta o render
```

O `memo` é o porteiro; o `useCallback` e o `useMemo` garantem que o que lhe chega é igual ao anterior. **Um sem o outro, quase sempre, não serve.**

As dependências

O `[ … ]` é a lista do que a função ou o valor **usam de fora**. Quando algo da lista muda, cria-se uma versão nova.

Esquecer um item → fica-se com um **valor antigo** (uma *stale closure*). O lint avisa (`exhaustive-deps`).

React Compiler

Um plugin de **build** que guarda em cache, dentro de cada componente, **valores, funções e JSX**, e só os recria quando aquilo de que dependem muda.

	Com o React Compiler
`useCallback` / `useMemo`	**Deixam de ser escritos** — o Compiler trata deles
`memo(Componente)`	Só onde o pai tem de **recriar** o elemento (ex.: itens de uma lista que depende de algo que mudou) **e** se mediu que pesa

E o código tem de ser **puro** (nada de `Math.random()`, `performance.now()` ou mudar variáveis de fora durante o render) — senão o Compiler salta esse componente.

Teste-se — 5 perguntas

1. Um filho sem `memo` recebe uma função com `useCallback`. Renderiza quando o pai renderiza?

Sim. Mas **não** por causa da função — com `useCallback` a referência **é a mesma**. Renderiza porque **não tem `memo`**: sem `memo`, um filho renderiza sempre que o pai renderiza e **nem olha para as props**.

Filho	Recebe	Renderiza quando o pai renderiza?
sem `memo`	função nova	**sim** — não compara nada
sem `memo`	função com `useCallback`	**sim** — o `useCallback` foi inútil
com `memo`	função nova	**sim** — compara, e a referência é outra
com `memo`	função com `useCallback`	**não** ✓

As props só contam quando há `memo`.

2. Um filho com `memo` recebe `style={{ color: 'red' }}`. Renderiza quando o pai renderiza?

Sim. `{{ color: 'red' }}` é um **objeto novo** em cada render. Soluções:

```
const redStyle = { color: 'red' };          // 1. constante FORA do componente (se nunca muda)
<Child color="red" />                        // 2. prop primitiva em vez de objeto
const style = useMemo(                       // 3. useMemo (se depende de state/props)
  () => ({ color: isError ? 'red' : 'black' }), [isError]);
```

3. `useMemo(() => a + b, [a, b])` vale a pena se `a` e `b` forem números pequenos?

Não. Somar dois números custa menos do que o próprio `useMemo` (guardar o valor e comparar dependências). E o resultado é um **primitivo**: `7 === 7`, não há referência a proteger.

`useMemo` só para cálculos pesados ou objetos/arrays que vão para um filho com `memo`.

4. Porque é que o `setBasket` não precisa de ir para as dependências do `useCallback`?

Duas razões:

1. O React garante que o `setBasket` é **sempre a mesma função**.
2. A **forma funcional** `setBasket(b => …)` recebe o valor atual — a função **não lê o `basket` de fora**:

```
// ✗ lê o basket de fora → o basket tem de ir para as deps → função nova a cada mudança
useCallback((id, sel) => setBasket({ ...basket, [id]: sel }), [basket]);

// ✓ forma funcional → deps vazias → sempre a mesma função
useCallback((id, sel) => setBasket(b => ({ ...b, [id]: sel })), []);
```

5. Com o React Compiler ligado, ainda escreveriam `useCallback` no `updatebasket`?

Não. O Compiler trata do `useCallback` e do `useMemo`. O que continua connosco:

- o **`memo`** onde o pai tem de recriar o elemento (como os cards dentro de um `map` que depende do `basket`) e se mediu que pesa;
- **código puro**, senão o Compiler salta o componente.

Para ler mais: memo · useMemo · useCallback · React Compiler — react.dev